SAS 2024

Should We Balance?

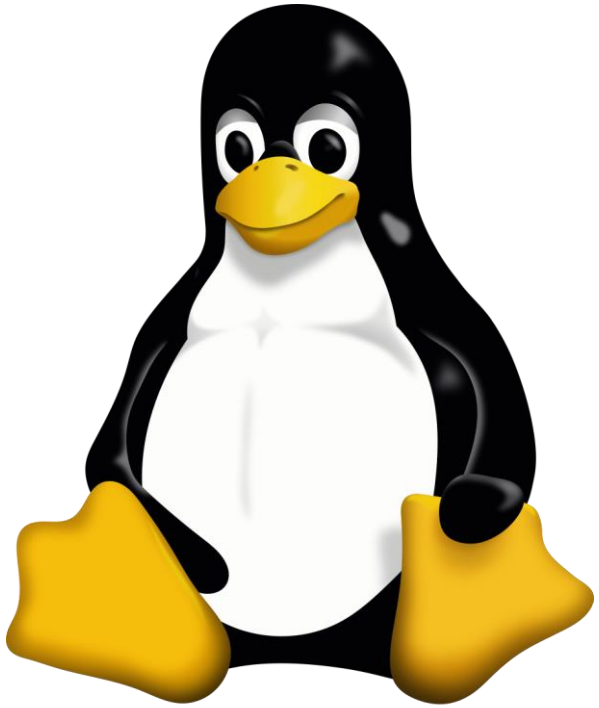
Towards Formal Verification of the Linux Kernel Scheduler

Julia Lawall, **Keisuke Nishimura,** Jean-Pierre Lozi



Background

The Linux kernel



- Seems reliable

- Reality: a lot of bugs
  - Common bug patterns (e.g., NULL Deref, Use-After-Free)
    - Detected by existing tools (e.g., Coccinelle)
  - Semantic bugs
    - No general methods for detection

Deductive verification can help identify semantic bugs.

Challenge of verification for Linux

- Verification for Linux is hard.
  - Written in C with heavy use of pointers and low-level features
  - Not designed for verification
    (c.f. verification friendly OSes such as SeL4, CertiKOS)

- A further wrinkle: Fast development = Frequent code updates



Is it possible to port specifications and proofs from one version to another?

Goal and Methods

Case study of verification of a function over versions

1. Provide specifications for a function.

2. Prove the correctness.

3. Try to port specifications and proofs over versions.

   How reusable?

   How hard to update?

Applying deductive verification

- A specification = Pre-conditions + Post-conditions
- Verifier: 

- No verification for callees
- No concurrency, no memory issues (e.g, NULL Deref)

The target function: should_we_balance()

- In a scheduler of the Linux kernel
- Decides if load balancing should be triggered.
  - Takes one CPU with its context
  - Returns true (trigger LB) or false (do nothing)
- Introduced in 2013, and maintained with 10 (+1 ours) versions

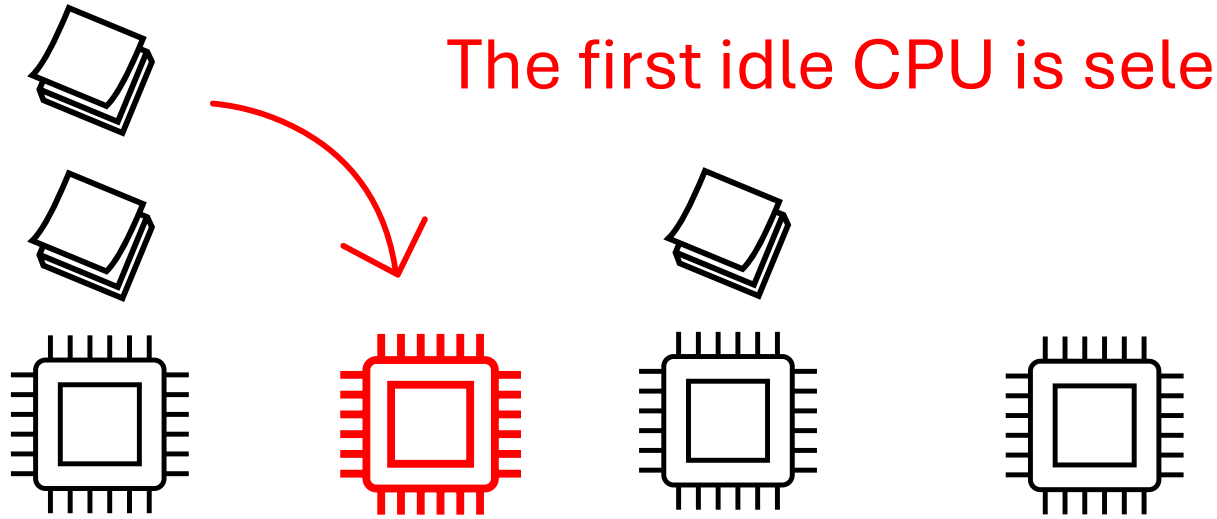
The target function: should_we_balance()

Load Balancing in Linux (simplified)

When there exists an idle CPU



CPU task



	CPU 0	CPU 1	CPU 2	CPU 3
status	Busy	Idle	-	Idle
should_we_balance	false	true	false	false

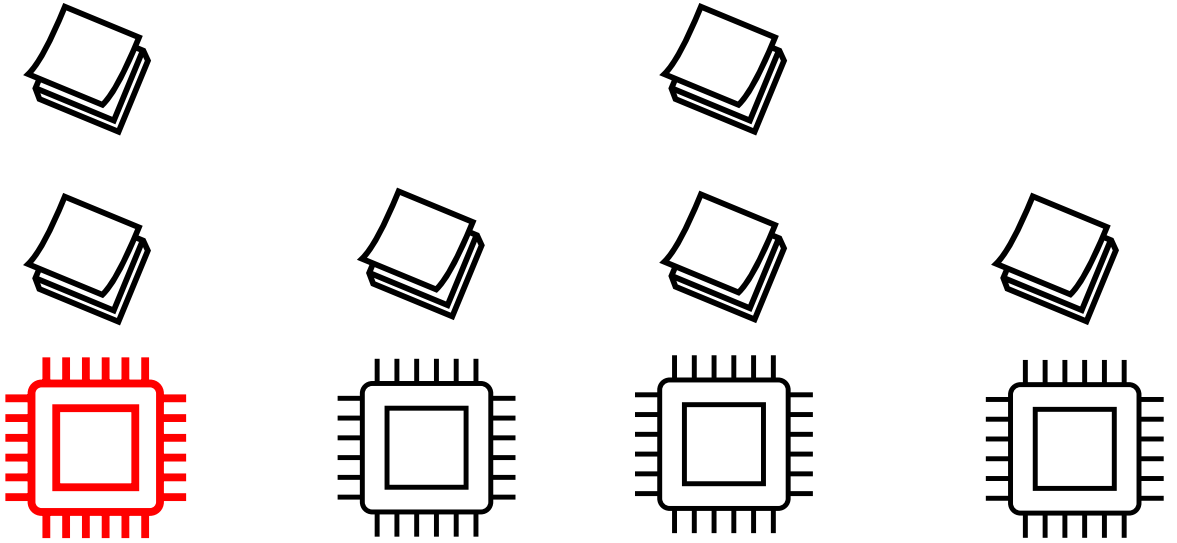
The target function: should_we_balance()

Load Balancing in Linux (simplified)

When no idle CPUs



CPU task



The first CPU is selected.

CPU 0 | CPU 1 | CPU 2 | CPU 3

	CPU 0	CPU 1	CPU 2	CPU 3
should_we_balance	true	false	false	false

The target function: should_we_balance()

Interesting Properties (simplified)

- Can be modeled as a predicate:

```
should_we_balance(cpu) = true/false
```

- Uniqueness

```
\forall cpu1,cpu2; cpu1 != cpu2 && should_we_balance(cpu1)
 => !should_we_balance(cpu2)
```

- Existence

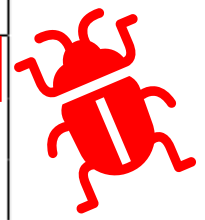
```
\exists cpu; should_we_balance(cpu)
```

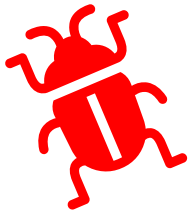
Brief history of should_we_balance()

#	Commit id	Date	Release	Impact
0	23f0d2093c78	Aug. 2013	–	create the function
1	b0cff9d88ce2	Sep. 2013	v3.12	replace != by ==
2	af218122b103	May 2017	–	eliminate a redundant function call
3	e5c14b1fb892	May 2017	v4.13	rename a function
4	024c9d2faebd	Oct. 2017	v4.14	check validity of the stealing CPU
5	97fb7a0a8944	Mar. 2018	v4.17	improve comments
6	64297f2b03cc	Apr. 2020	v5.8	return early on finding an idle core
7	792b9f65a568	Jun. 2022	v6.0	bail out if tasks are detected on a newly idle CPU
8	b1bfeab9b002	Jul. 2023	–	prefer fully idle cores
9	f8858d96061f	Sep. 2023	v6.6	remove non-idle core hyperthreads from the CPU mask
10	6d7e4782bcf5	Oct. 2023	v6.8	change a condition of the selection algorithm

Brief history of should_we_balance()

#	Commit id	Date	Release	Impact
0	23f0d2093c78	Aug. 2013	–	create the function
1	b0cff9d88ce2	Sep. 2013	v3.12	replace != by ==
2	af218122b103	May 2017	–	eliminate a redundant function call
3	e5c14b1fb892	May 2017	v4.13	rename a function
4	024c9d2faebd	Oct. 2017	v4.14	check validity of the stealing CPU
5	97fb7a0a8944	Mar. 2018	v4.17	improve comments
6	64297f2b03cc	Apr. 2020	v5.8	return early on finding an idle core
7	792b9f65a568	Jun. 2022	v6.0	bail out if tasks are detected on a newly idle CPU
8	b1bfeab9b002	Jul. 2023	–	prefer fully idle cores
9	f8858d96061f	Sep. 2023	v6.6	remove non-idle core hyperthreads from the CPU mask
10	6d7e4782bcf5	Oct. 2023	v6.8	change a condition of the selection algorithm





Implementation of should_we_balance()

```
static int should_we_balance(struct lb_env *env)
{
        [...]
        int cpu, balance_cpu = -1;
        [...]
        for_each_cpu_and(cpu, sg_cpus, env->cpus) {
                if (...|| !idle_cpu(cpu))
                        continue;
                balance_cpu = cpu;
                break;
        }
        if (balance_cpu == -1)
                balance_cpu = group_balance_cpu(sg);
        return balance_cpu == env->dst_cpu;
}
```

dst_cpu is the target CPU

Is the given CPU balance_cpu?

Specification of should_we_balance()

```
behavior not_newly_idle1 :
  . . .
  assumes \exists integer i; relevant(i, env) && idle_cpu(i);
  ensures \forall integer i;
   relevant(i, env) ==> idle_cpu(i) ==>
   (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_cpu(j)) ==>
   (\result <==> env->dst_cpu == i);

behavior not_newly_idle2 :
  . . .
  assumes \forall integer i; relevant(i, env) ==> !idle_cpu(i);
  ensures \result <==> group_balance_cpu(env->sd->groups) == env->dst_cpu;
```

Case-study:

Specification update after optimization

#	Commit id	Date
0	23f0d2093c78	Aug. 2013
1	b0cff9d88ce2	Sep. 2013
2	af218122b103	May 2017
3	e5c14b1fb892	May 2017
4	024c9d2faebd	Oct. 2017
5	97fb7a0a8944	Mar. 2018
6	64297f2b03cc	Apr. 2020
7	792b9f65a568	Jun. 2022
8	b1bfeab9b002	Jul. 2023
9	f8858d96061f	Sep. 2023
10	6d7e4782bcf5	Oct. 2023

Optimization with early-return

Introduced by 64297f2b03cc in April 2020

```
        for_each_cpu_and(cpu, sg_cpus, env->cpus) {
                if (!idle_cpu(cpu))
                        continue;
-               balance_cpu = cpu;
-               break;
+               return cpu == env->dst_cpu;
        }
-       if (balance_cpu == -1)
-               balance_cpu = group_balance_cpu(sg);
-        return balance_cpu == env->dst_cpu;
+       return group_balance_cpu(sg) == env->dst_cpu;
```

Optimization with early-return

```
behavior not_newly_idle1 :
  . . .
  assumes \exists integer i; relevant(i, env) && idle_cpu(i);
  ensures \forall integer i;
   relevant(i, env) ==> idle_cpu(i) ==>
   (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_cpu(j)) ==>
   (\result <==> env->dst_cpu == i);

behavior not_newly_idle2 :
  . . .
  assumes \forall integer i; relevant(i, env) ==> !idle_cpu(i);
  ensures \result <==> group_balance_cpu(env->sd->groups) == env->dst_cpu;
```

No update in the specifications and the proofs

Case-study:

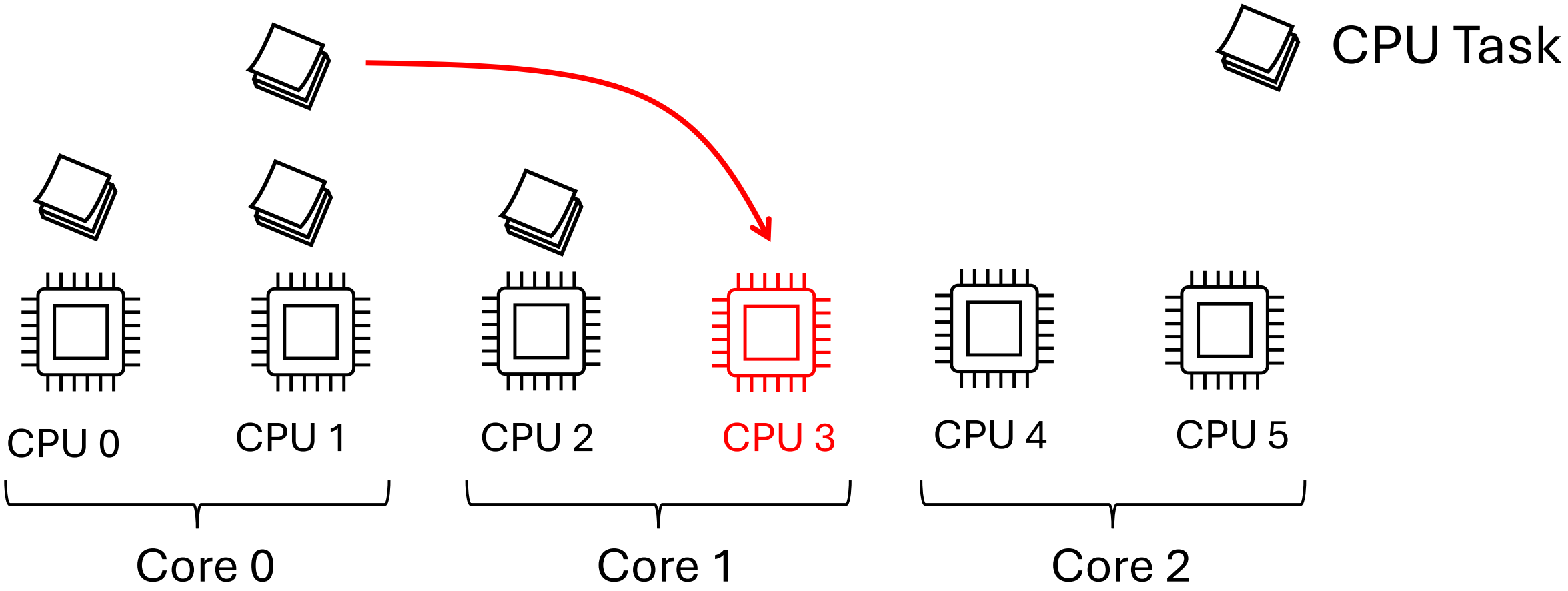
Specification update
after an algorithm change



#	Commit id	Date
0	23f0d2093c78	Aug. 2013
1	b0cff9d88ce2	Sep. 2013
2	af218122b103	May 2017
3	e5c14b1fb892	May 2017
4	024c9d2faebd	Oct. 2017
5	97fb7a0a8944	Mar. 2018
6	64297f2b03cc	Apr. 2020
7	792b9f65a568	Jun. 2022
8	b1bfeab9b002	Jul. 2023
9	f8858d96061f	Sep. 2023
10	6d7e4782bcf5	Oct. 2023

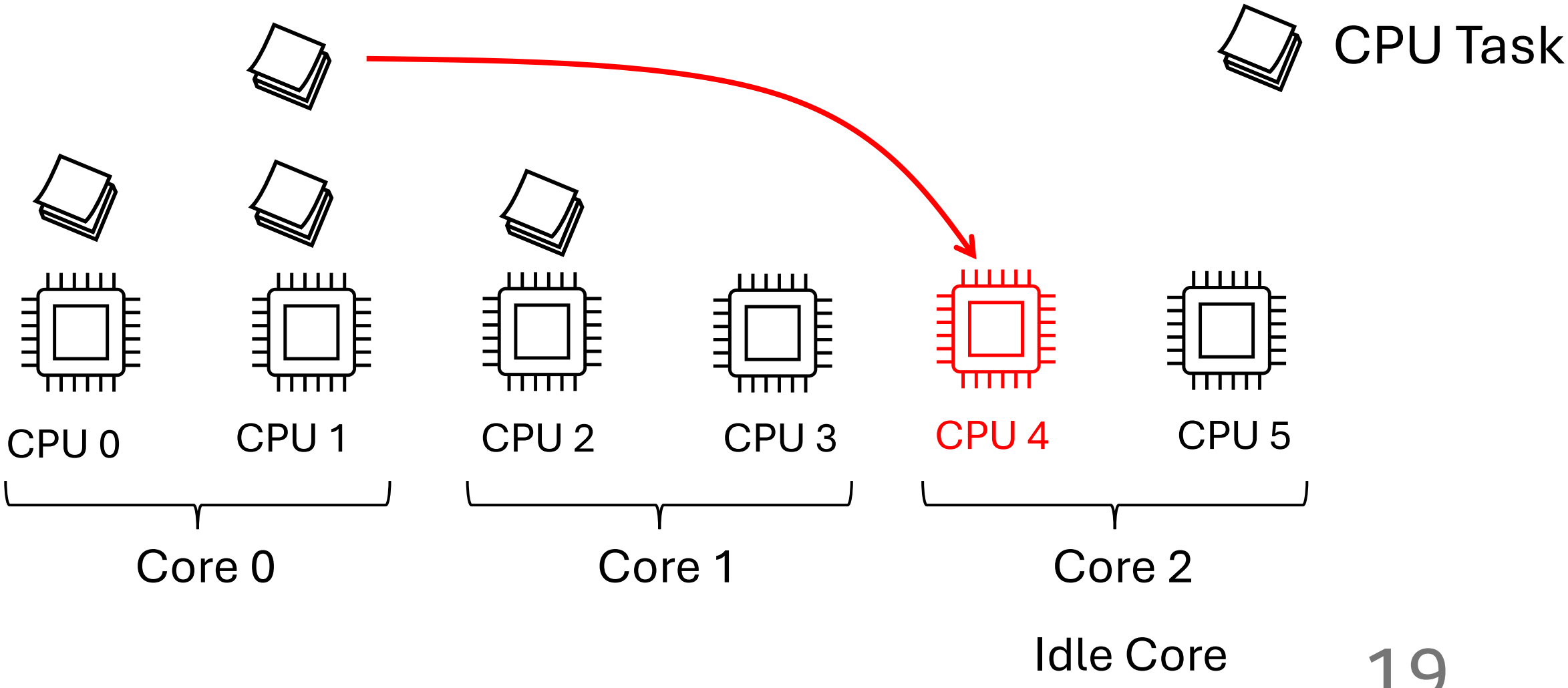
Algorithm change in load balancing

Previous: idle && first CPU



Algorithm change in load balancing

New: considering SMT (Simultaneous Multi-Threading)



New algorithm in b1bfeab9b00

Change of code

```
        for_each_cpu_and(cpu, sg_cpus, env->cpus) {
           if (!idle_cpu(cpu))
               continue;
+          if (... && !is_core_idle(cpu)) {
+             if (idle_smt == -1)
+                idle_smt = cpu;
+             continue;
+           }
            return cpu == env->dst_cpu;
        }
+       if (idle_smt == env->dst_cpu)
+          return true;
        return group_balance_cpu(sg) == env->dst_cpu;
```

New algorithm in b1bfeab9b00

Change of loop invariants

```
/*@
  loop invariant 0 <= cpu <= small_cpumask_bits;
-  loop invariant \forall integer j; 0 <= j < cpu ==> relevant(j, env) ==> !idle_cpu(j);
-  loop assigns cpu;
   ...
+  loop invariant idle_smt == -1 ==> \forall integer j; 0 <= j < cpu ==> relevant(j, env)
   ==> !idle_cpu(j);
+  loop invariant idle_smt != -1 ==> 0 <= idle_smt < cpu && relevant(idle_smt, env) &&
   idle_cpu(idle_smt);
+  loop invariant idle_smt != -1 ==> \forall integer j; 0 <= j < idle_smt ==>
   relevant(j, env) ==> !idle_cpu(j);
+  loop invariant idle_smt != -1 ==> \forall integer j; idle_smt <= j < cpu ==>
  relevant(j, env) ==> !idle_core(j);
+  loop assigns cpu , idle_smt;
  loop variant small_cpumask_bits - cpu;
 */
```

Implementation of the new algorithm

Change of specifications

```
behavior not_newly_idle_with_idle:
  assumes env->idle != CPU_NEWLY_IDLE;
  assumes env->cpus->bits[env->dst_cpu];
  assumes \exists integer i; relevant(i, env) && idle_cpu(i);
  ensures \forall integer i;
    relevant(i, env) ==> idle_cpu(i) ==>
    (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_cpu(j)) ==>
    (\result <==> env->dst_cpu == i);
  ensures \forall integer i;
    \at(relevant(i, env) && idle_cpu(i) &&
    (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_cpu(j)),Pre) ==>
    (\result <==> \at(env->dst_cpu == i,Pre));

behavior not_newly_idle_without_idle:
  assumes env->idle != CPU_NEWLY_IDLE;
  assumes env->cpus->bits[env->dst_cpu];
  assumes \forall integer i; relevant(i, env) ==> !idle_cpu(i);
  ensures \result <==> group_balance_cpu(env->sd->groups) == env->dst_cpu;
  ensures \result <==> \at(group_balance_cpu(env->sd->groups) == env->dst_cpu,Pre);
```

More than double lines of code

```
behavior not_newly_idle_with_idle:
  assumes env->idle != CPU_NEWLY_IDLE;
  assumes env->cpus->bits[env->dst_cpu];
  assumes env->sd->flags & SD_SHARE_CPUCAPACITY;
  assumes \exists integer i; relevant(i, env) && idle_cpu(i);
  ensures \forall integer i;
    relevant(i, env) ==> idle_cpu(i) ==>
    (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_cpu(j)) ==>
    (\result <==> env->dst_cpu == i);
  ensures \forall integer i;
    \at(relevant(i, env) && idle_cpu(i) &&
    (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_cpu(j)),Pre) ==>
    (\result <==> \at(env->dst_cpu == i,Pre));

behavior not_newly_idle_with_idle_core:
  assumes env->idle != CPU_NEWLY_IDLE;
  assumes env->cpus->bits[env->dst_cpu];
  assumes !(env->sd->flags & SD_SHARE_CPUCAPACITY);
  assumes \exists integer i; relevant(i, env) && idle_core(i);
  ensures \forall integer i;
    relevant(i, env) ==> idle_core(i) ==>
    (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_core(j)) ==>
    (\result <==> env->dst_cpu == i);
  ensures \forall integer i;
    \at(relevant(i, env) && idle_core(i) &&
    (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_core(j)),Pre) ==>
    (\result <==> \at(env->dst_cpu == i,Pre));


behavior not_newly_idle_with_idle_cpu:
  assumes env->idle != CPU_NEWLY_IDLE;
  assumes env->cpus->bits[env->dst_cpu];
  assumes !(env->sd->flags & SD_SHARE_CPUCAPACITY);
  assumes \forall integer i; relevant(i, env) ==> !idle_core(i);
  assumes \exists integer i; relevant(i, env) && idle_cpu(i);
  ensures \forall integer i;
    relevant(i, env) ==> idle_cpu(i) ==>
    (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_cpu(j)) ==>
    (\result ==> (env->dst_cpu == i || env->dst_cpu == group_balance_cpu(env->sd->groups)));
  ensures \forall integer i;
    relevant(i, env) ==> idle_cpu(i) ==>
    (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_cpu(j)) ==>
    (env->dst_cpu == i ==> \result);
  ensures \forall integer i;
    \at(relevant(i, env) && idle_cpu(i) &&
    (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_cpu(j)),Pre) ==>
    (\result ==> \at((env->dst_cpu == i || env->dst_cpu == group_balance_cpu(env->sd->groups)),Pre
  ensures \forall integer i;
    \at(relevant(i, env) && idle_cpu(i) &&
    (\forall integer j; 0 <= j < i ==> relevant(j, env) ==> !idle_cpu(j)),Pre) ==>
    (\at(env->dst_cpu == i,Pre) ==> \result);

behavior not_newly_idle_without_idle:
  assumes env->idle != CPU_NEWLY_IDLE;
  assumes env->cpus->bits[env->dst_cpu];
  assumes \forall integer i; relevant(i, env) ==> !idle_cpu(i);
  ensures \result <==> group_balance_cpu(env->sd->groups) == env->dst_cpu;
  ensures \result <==> \at(group_balance_cpu(env->sd->groups) == env->dst_cpu,Pre);
```

Case-study:

A bug found in the function

#	Commit id	Date
0	23f0d2093c78	Aug. 2013
1	b0cff9d88ce2	Sep. 2013
2	af218122b103	May 2017
3	e5c14b1fb892	May 2017
4	024c9d2faebd	Oct. 2017
5	97fb7a0a8944	Mar. 2018
6	64297f2b03cc	Apr. 2020
7	792b9f65a568	Jun. 2022
8	b1bfeab9b002	Jul. 2023
9	f8858d96061f	Sep. 2023
10	6d7e4782bcf5	Oct. 2023

A bug introduced in the new algorithm

Original implementation

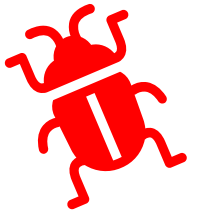
```
        for_each_cpu_and(cpu, sg_cpus, env->cpus) {
                if (!idle_cpu(cpu))
                        continue;
                if (... && !is_core_idle(cpu)) {
                        if (idle_smt == -1) {
                                idle_smt = cpu;
                        continue;
                }
                return cpu == env->dst_cpu;
        }
         if (idle_smt == env->dst_cpu)
                return true;
        return group_balance_cpu(sg) == env->dst_cpu;
```

A bug introduced in the new algorithm

Original implementation

```
for_each_cpu_and(cpu, sg_cpus, env->cpus) {
        if (!idle_cpu(cpu))
                continue;
        if (... && !is_core_idle(cpu)) {
                if (idle_smt == -1) {
                        idle_smt = cpu;
                continue;
        }
        return cpu == env->dst_cpu;
}
 if (idle_smt == env->dst_cpu)
        return true;
return group_balance_cpu(sg) == env->dst_cpu;
```

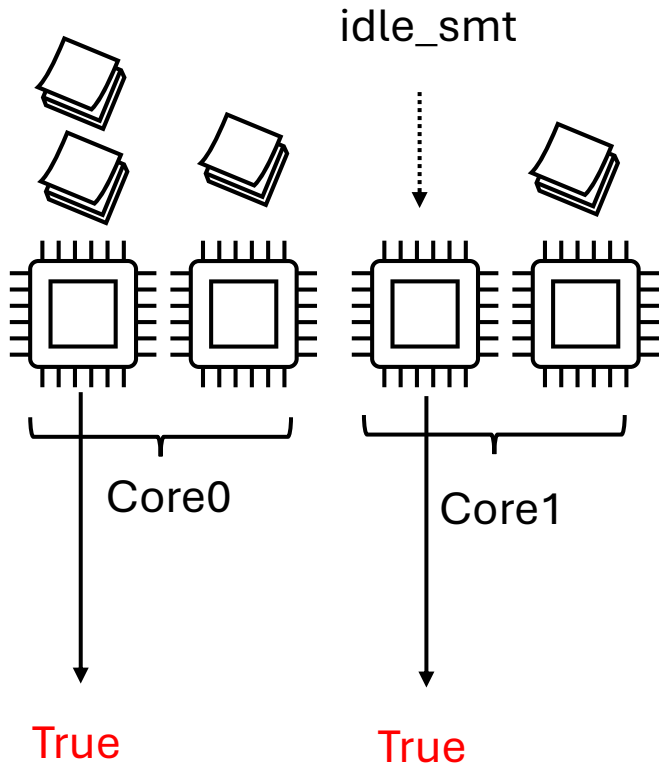
BUG!



Otherwise, assume that there are no CPU-idle CPUs

Details of the bug

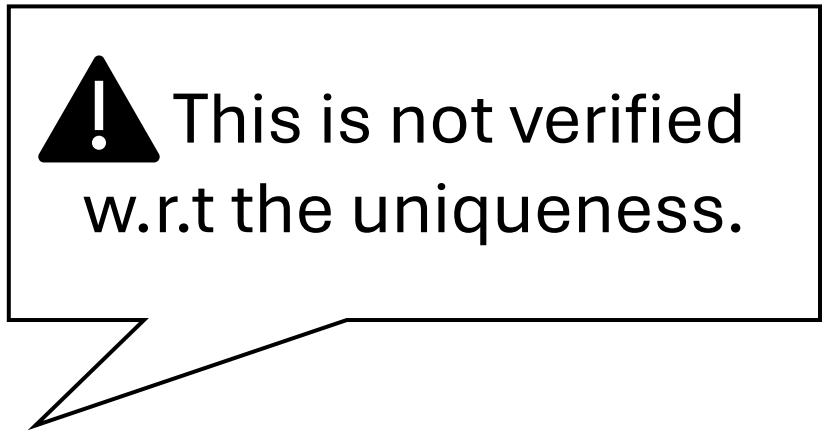
```
for_each_cpu_and(cpu, sg_cpus, env->cpus) {
        if (!idle_cpu(cpu))
                continue;
        if (... && !is_core_idle(cpu)) {
                if (idle_smt == -1) {
                        idle_smt = cpu;
                continue;
        }
        return cpu == env->dst_cpu;
}
 if (idle_smt == env->dst_cpu)
        return true;
return group_balance_cpu(sg) == env->dst_cpu;
```



Violation of the uniqueness property

Bug fix proposed by us (simplified)

```
        for_each_cpu_and(cpu, sg_cpus, env->cpus) {
                if (!idle_cpu(cpu))
                        continue;
                if (... && !is_core_idle(cpu)) {
                        if (idle_smt == -1) {
                                idle_smt = cpu;
                        continue;
                }
                return cpu == env->dst_cpu;
        }
-       if (idle_smt == env->dst_cpu)
-               return true;
+       if (idle_smt == -1)
+               return idle_smt == env->dst_cpu;
        return group_balance_cpu(sg) == env->dst_cpu;
```



Bug fix proposed by us

author	Keisuke Nishimura <keisuke.nishimura@inria.fr>	2023-10-31 14:38:22 +0100
committer	Peter Zijlstra <peterz@infradead.org>	2023-11-14 22:27:01 +0100
commit	6d7e4782bcf549221b4ccfffec2cf4d1a473f1a3 (patch)	
tree	fec2d1d9b8e4c1c2f5fb3e0a9255e016c93ba5b3	
parent	8b39d20eceeda6c4eb23df1497f9ed2fffdc8f69 (diff)	
download	linux-6d7e4782bcf549221b4ccfffec2cf4d1a473f1a3.tar.gz	

sched/fair: Fix the decision for load balance

▪ ▪ ▪

Merged in upstream 

```
diff --git a/kernel/sched/fair.c b/kernel/sched/fair.c
index 025d90925bf631..d7a3c63a2171a8 100644
--- a/kernel/sched/fair.c
+++ b/kernel/sched/fair.c
@@ -11184,12 +11184,16 @@ static int should_we_balance(struct lb_env *env)
                        continue;
                }

-               /* Are we the first idle CPU? */
+               /*
+                * Are we the first idle core in a non-SMT domain or higher,
+                * or the first idle CPU in a SMT domain?
+                */
                return cpu == env->dst_cpu;
        }

-       if (idle_smt == env->dst_cpu)
-               return true;
+       /* Are we the first idle CPU with busy siblings? */
+       if (idle_smt != -1)
+               return idle_smt == env->dst_cpu;

        /* Are we the first CPU of this group ? */
        return group_balance_cpu(sg) == env->dst_cpu;
```

More details in the paper

- Other updates and their effect on specifications
- Required time for verification
- Other difficulties in verification (especially, about pointers)

…

Lessons Learned

In the case of should_we_balance()...

- No need to update specification for some code change including optimization.
- Lots of work needed to update the specification for a new algorithm.
- Bugs can be found in the kernel.

Future Work

- Verification of the uniqueness/existence properties for all versions
- Verification of more functions over versions
- Automation of porting specifications
- Developer-friendly verification process

Conclusion

- Case study: verification of should_we_balance() across 11 versions

- Key points
  - Optimizations: Often compatible with existing spec.
  - Algorithm Changes: Require significant spec. updates.
  - Bug Detection: Verification helps uncover issues.

https://zenodo.org/records/13132904

#	Commit id	Impact	Change type	Proof impact
0	23f0d2093c78	create the function	new algorithm	new specification
1	b0cff9d88ce2	replace `!=` by `==`	bug fix	invariant update
2	af218122b103	eliminate a redundant function call	refactoring	invariant update
3	e5c14b1fb892	rename a function	refactoring	invariant update
4	024c9d2faebd	check validity of the stealing CPU	refinement	new `behavior`
5	97fb7a0a8944	improve comments	refactoring	none
6	64297f2b03cc	return early on finding an idle core	refactoring	none
7	792b9f65a568	bail out if tasks are detected on a newly idle CPU	refinement	new `behavior`
8	b1bfeab9b002	prefer fully idle cores	new algorithm	many changes
9	f8858d96061f	remove non-idle core hyperthreads from the CPU mask	new algorithm	many changes
10	6d7e4782bcf5	change a condition of the selection algorithm	bug fix	invariant update

	Without smoke			With smoke		
#	VCs proved	VC max time	total time	VCs proved	VC max time	total time
0	256	0.398-0.401s	39.99-40.13s	343	0.412-0.443s	2:32-2:33m
1	256	0.378-0.419s	39.82-40.22s	343	0.433-0.439s	2:39-2:40m
2	250	0.370-0.413s	39.27-39.31s	333	0.395-0.421s	2:24-2:25m
3	251	0.376-0.383s	39.23-39.48s	334	0.405-0.425s	2:25m
4	258	0.451-0.457s	39.53-39.77s	344	0.457-0.478s	2:31-2:38m
5	258	0.446-0.496s	39.78-40.57s	344	0.443-0.453s	2:31-3:15m
6	257	0.400-0.408s	39.57-39.66s	343	0.405-0.431s	2:31-2:37m
7	263	0.409-0.431s	40.68-41.29s	351	0.417-0.439s	2:35-2:36m
8	288	0.516-0.562s	50.74-51.03s	382	0.581-0.634s	2:56-2:57m
9u	301	214-247s	9:30-10:19m	397	221-258s	11:51-12:42m
9d	314	276-466s	9:26-10:24m	414	245-343s	11:25-12:22m
10u	301	123-206s	7:25-8:59m	397	112-203s	9:25-11:11m
10d	314	160-357s	8:44-11:22m	414	233-275s	12:20-13:10m
10au	299	169-194s	6:46-7:04m	395	110-221s	7:53-9:48m
10ad	312	208-255s	9:01-11:30m	412	235-343s	12:10-14:48m

```
1 + struct cpumask *swb_cpus =
2 +   this_cpu_cpumask_var_ptr(should_we_balance_tmpmask);
3   [...]
4 - for_each_cpu_and(cpu, group_balance_mask(sg), env->cpus) {
5 + cpumask_copy(swb_cpus, group_balance_mask(sg));
6 + for_each_cpu_and(cpu, swb_cpus, env->cpus) {
7     if (!idle_cpu(cpu))
8       continue;
9     if (!(env->sd->flags & SD_SHARE_CPUCAPACITY) &&
10        !is_core_idle(cpu)) {
11      if (idle_smt == -1)
12        idle_smt = cpu;
13 +#ifdef CONFIG_SCHED_SMT
14 +     cpumask_andnot(swb_cpus, swb_cpus, cpu_smt_mask(cpu));
15 +#endif
16      continue;
17    }
18    return cpu == env->dst_cpu;
19  }
```